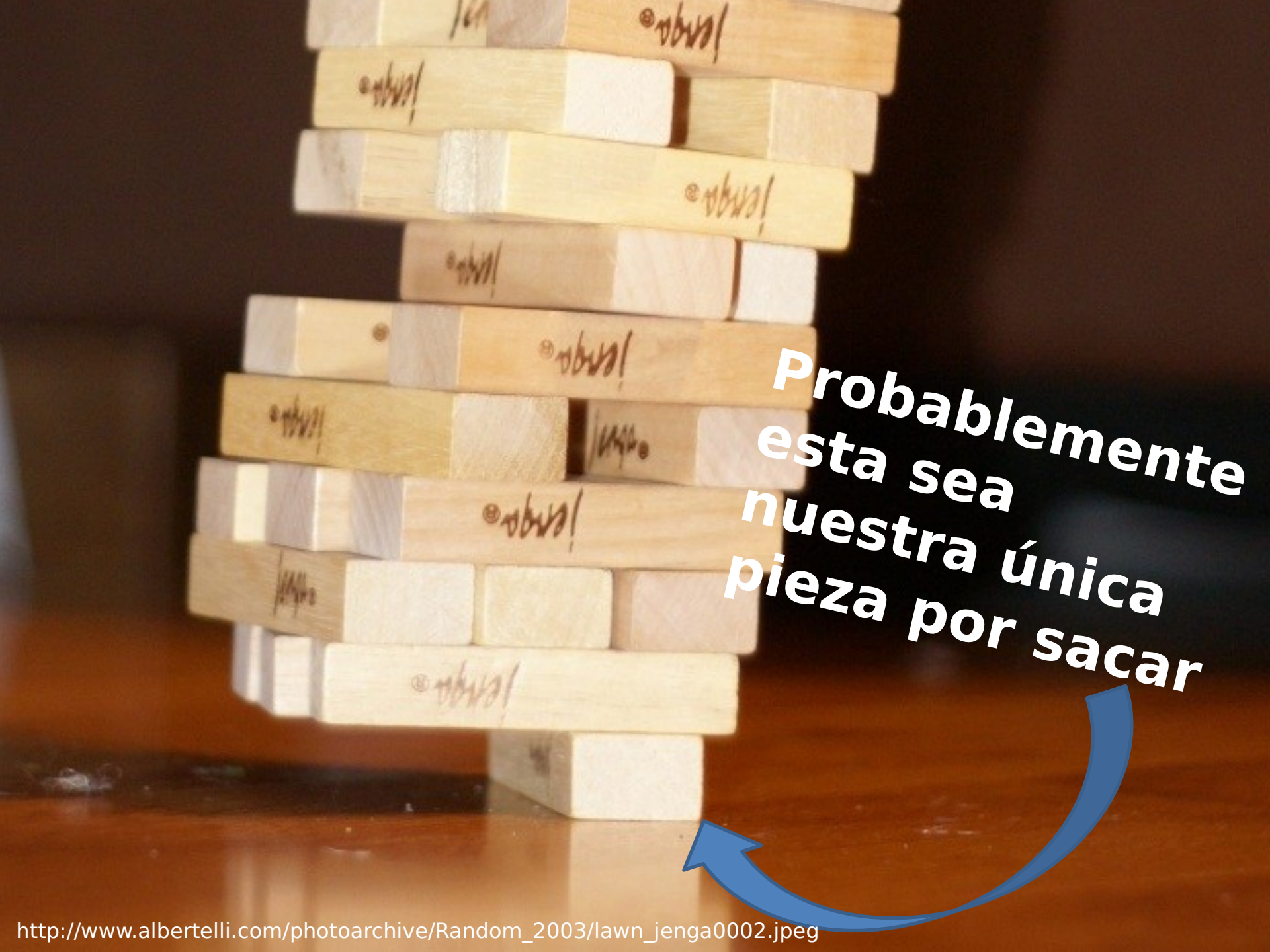


S.O.L.I.D.

¿Cómo lo aplico en mi código?



**Que pasa
cuando nos
toca
modificar
código?**



**Probablemente
esta sea
nuestra única
pieza por sacar**

S.O.L.I.D. es un acrónimo de:

- **S**ingle Responsibility
- **O**pen Closed
- **L**iskov Substitution
- **I**nterface Segregation
- **D**ependency Inversion



SINGLE RESPONSIBILITY PRINCIPLE

Just Because You Can, Doesn't Mean You Should

Single Responsibility Principle

Una clase jamás debería tener más de una razón por la cual cambiar

- Responsabilidad == Razón para cambiar
- Si una clase asume más de una responsabilidad, entonces tendrá más de una razón para cambiar.
- Acoplamiento de responsabilidades.

Single Responsibility Principle

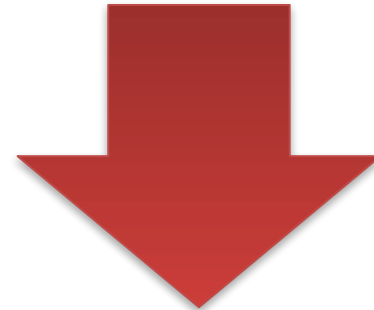
Cohesión:

Qué tan fuertemente relacionadas y enfocadas están las distintas responsabilidades de un módulo.

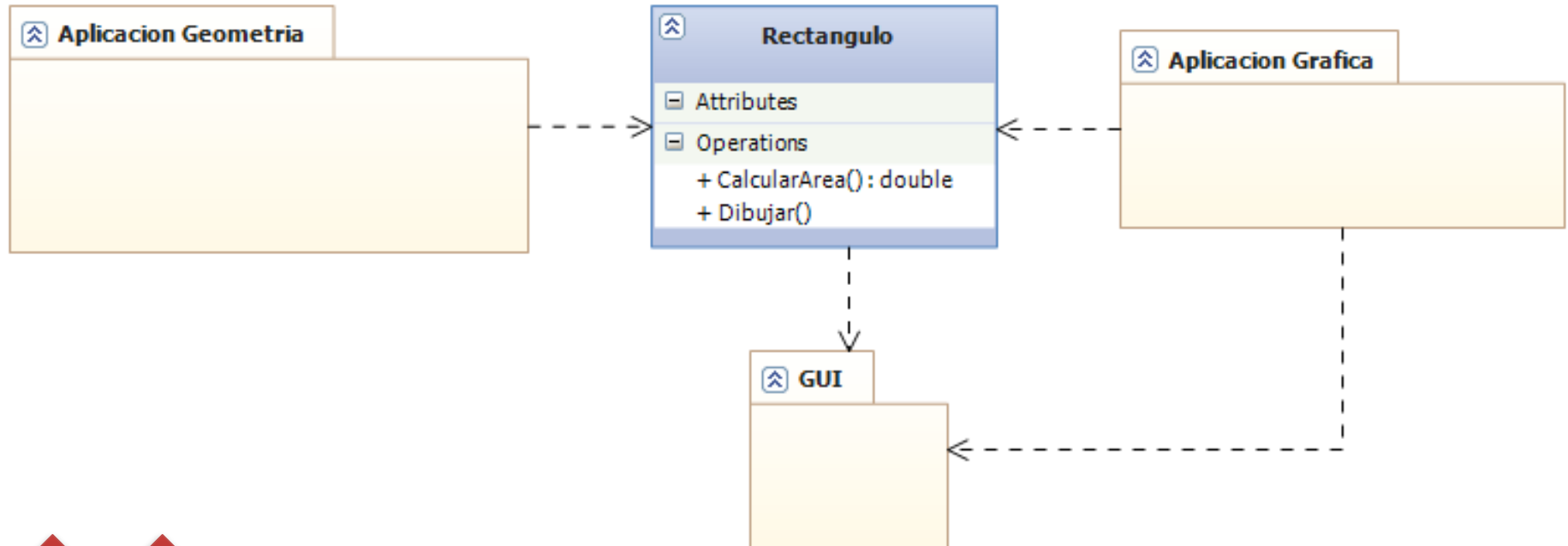


Acoplamiento:

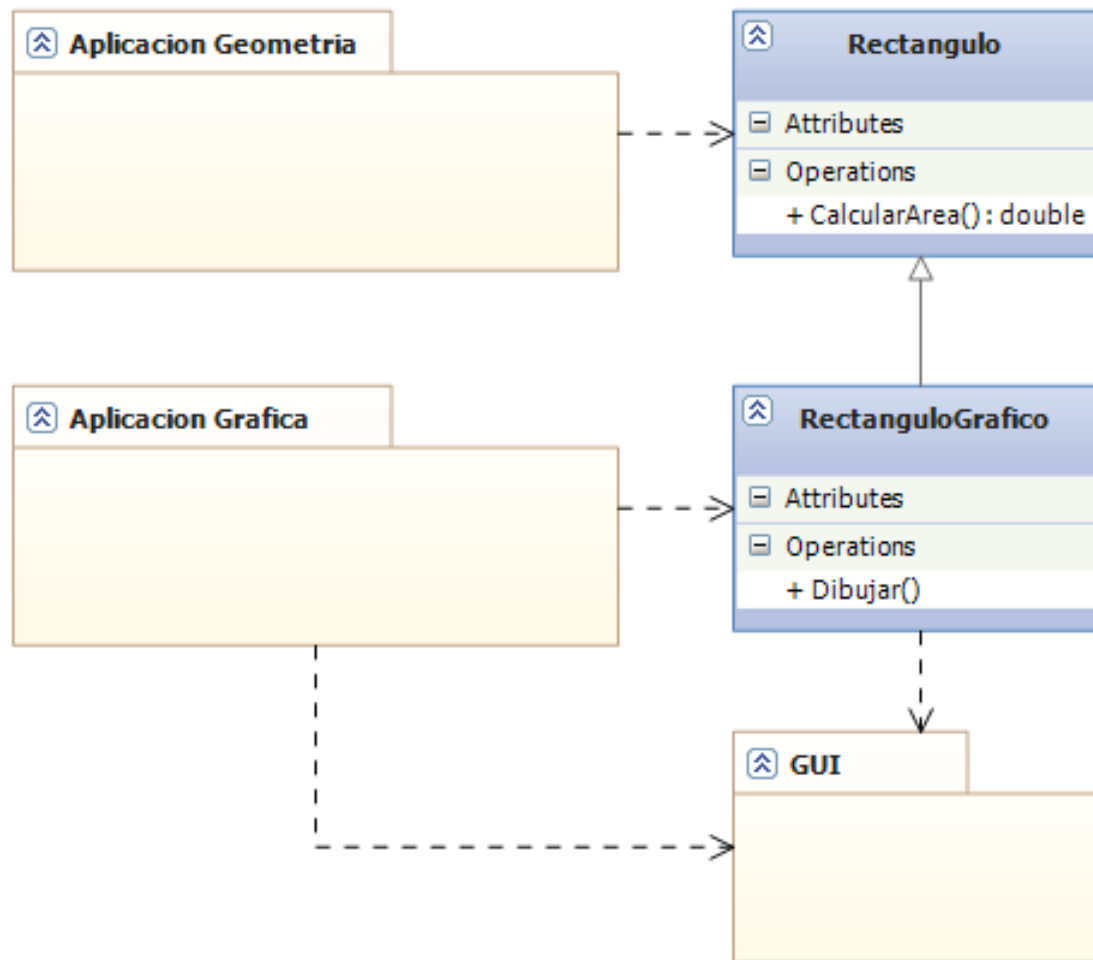
El grado en el cual cada módulo de un programa depende de cada uno de los otros módulos



Single Responsibility Principle



Single Responsibility Principle





OPEN CLOSED PRINCIPLE

Open Chest Surgery Is Not Needed When Putting On A Coat

Open Closed Principle

Entidades de software (clases, módulos, funciones, etc.) deberían estar abiertas para extensión pero cerradas para modificación.

- Si **1** cambio impacta a varios módulos, entonces la aplicación no está bien diseñada.
- Debemos diseñar módulos que nunca cambien

Open Closed Principle

**Abiertas para
extensión**

Podemos hacer que la aplicación se comporte de distintas formas.

Extendiendo el comportamiento del módulo.

**Cerradas para
modificación**

No se necesita hacer cambios del código fuente de dicho módulo.

Pero cómo?

Abstracción

Open Closed Principle

```
public enum TipoEmpleado
{
    Programador,
    Gerente
}

public class Empleado
{
    public string Nombre { get; set; }
    public double Sueldo { get; set; }
    public double Bono { get; set; }
    public TipoEmpleado Tipo { get; set; }
}
```

```
public class EmpleadosServicio
{
    public List<Empleado> Empleados { get; set; }

    public void CalcularBonos()
    {
        foreach (var empleado in Empleados)
        {
            double bono = 0;

            switch (empleado.Tipo)
            {
                case TipoEmpleado.Programador:
                    bono = empleado.Sueldo * 2;
                    break;
                case TipoEmpleado.Gerente:
                    bono = empleado.Sueldo * 10;
                    break;

                // Estamos contratando
                // analistas funcionales!!
            }

            empleado.Bono = bono;
        }
    }
}
```



Open Closed Principle

```
public abstract class Empleado
{
    public string Nombre { get; set; }
    public double Sueldo { get; set; }
    public double Bono { get; set; }

    public abstract void CalcularBono();
}
```

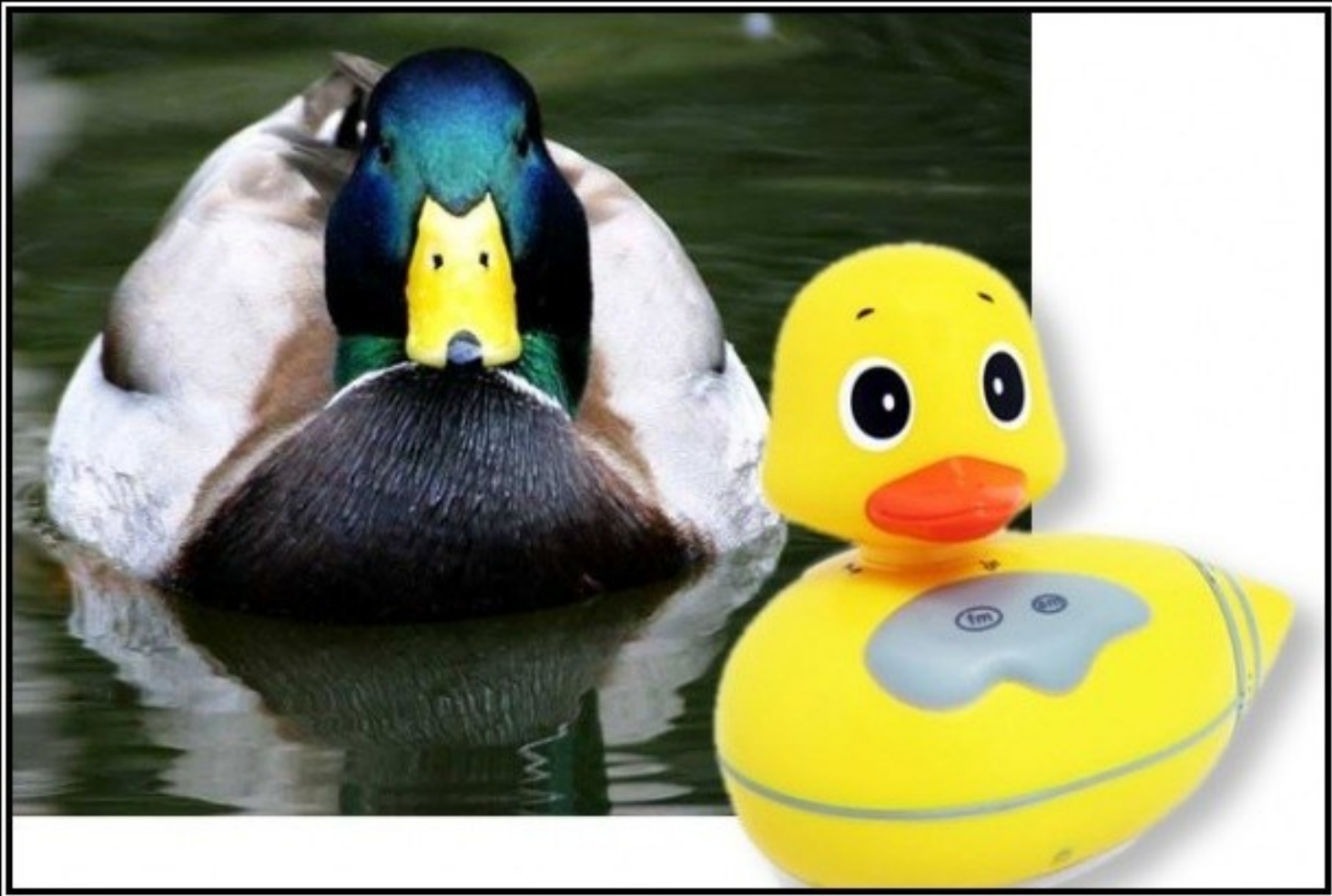
```
public class Programador : Empleado
{
    public override void CalcularBono()
    {
        Bono = Sueldo * 2;
    }
}
```

```
public class Gerente : Empleado
{
    public override void CalcularBono()
    {
        Bono = Sueldo * 15;
    }
}
```

```
public class EmpleadosServicio
{
    public List<Empleado> Empleados { get; set; }

    public void CalcularBonos()
    {
        foreach (var empleado in Empleados)
        {
            empleado.CalcularBono();
        }
    }
}
```





LISKOV SUBSTITUTION PRINCIPLE

If It Looks Like A Duck, Quacks Like A Duck, But Needs Batteries - You Probably Have The Wrong Abstraction

Liskov Substitution Principle

Funciones que usen punteros o referencias a clases base deben poder usar objetos de clases derivadas sin saberlo.

- Si tenemos una clase **BASE** y dos subclases **SUB1** y **SUB2**, el código cliente siempre debe referirse a **BASE**.
- No decir: **SUB1** *es una* **BASE**.
- En cambio decir: **SUB1** *es reemplazable por una* **BASE**.

Liskov Substitution Principle

```
foreach (var empleado in Empleados)
{
    if (empleado is Programador)
    {
        empleado.Bono = empleado.Sueldo * 2;
    }
    else if (empleado is Gerente)
    {
        empleado.Bono = empleado.Sueldo * 10;
    }
}
```



```
public void CalcularBonos()
{
    foreach (var empleado in Empleados)
    {
        empleado.CalcularBono();
    }
}
```





INTERFACE SEGREGATION PRINCIPLE

You Want Me To Plug This In, Where?

Interface Segregation Principle

Los clientes no deberían estar forzados a depender de interfaces que no utilizan.

- Las interfaces “gordas” o “contaminadas” deben dividirse en varios grupos de funciones.
- Cada grupo será implementado por distintos tipos de clientes.

Interface Segregation Principle

```
public class Perro : Animal
{
    public override void Alimentar()
    {
        // Alimentar al perro
    }

    public override void Acariciar()
    {
        // Acariciar al perro
    }
}

public abstract class Animal
{
    public abstract void Alimentar();
    public abstract void Acariciar();
}

public class Escorpion : Animal
{
    public override void Alimentar()
    {
        // Alimentar al escorpion
    }

    public override void Acariciar()
    {
        // Estas loco ????!
    }
}
```



Interface Segregation Principle

```
public abstract class Animal
{
    public abstract void Alimentar();
}

public interface IMascota
{
    void Acariciar();
}
```

```
public class Perro : Animal, IMascota
{
    public override void Alimentar()
    {
        // Alimentar al perro
    }

    public void Acariciar()
    {
        // Acariciar al perro
    }
}
```

```
public class Escorpion : Animal
{
    public override void Alimentar()
    {
        // Alimentar al escorpion
    }
}
```





DEPENDENCY INVERSION PRINCIPLE

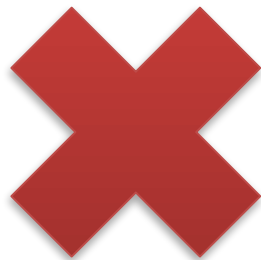
Would You Solder A Lamp Directly To The Electrical Wiring In A Wall?

Dependency Inversion Principle

- Módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones.
- Abstracciones no deben depender de detalles. Los detalles deben depender de abstracciones.
- Puede implementarse con:
 - Inyección de dependencias
 - IoC (Inversión del control)

DIP - Ejemplo 1

```
public class HolaMundo
{
    public string Saludar(string nombre)
    {
        if (DateTime.Now.Hour < 12)
            return "Buenos dias " + nombre;
        if (DateTime.Now.Hour < 19)
            return "Buenas tardes " + nombre;
        return "Buenas noches " + nombre;
    }
}
```



<https://gist.github.com/>

```
public class HolaMundo
{
    private DateTime _horaDelSaludo;

    public HolaMundo(DateTime horaDelSaludo)
    {
        _horaDelSaludo = horaDelSaludo;
    }

    public string Saludar(string nombre)
    {
        if (_horaDelSaludo.Hour < 12)
            return "Buenos días " + nombre;
        if (_horaDelSaludo.Hour < 19)
            return "Buenas tardes " + nombre;
        return "Buenas noches " + nombre;
    }
}
```



<https://gist.github.com/>

DIP – Ejemplo 2

```
public class Volvo
{
    private B20 _motor;

    public Volvo()
    {
        _motor = new B20();
    }
}
```

```
public class Volvo
{
    private IMotor _motor;

    public Volvo(IMotor motor)
    {
        _motor = motor;
    }
}
```

```
var miVolvo = new Volvo(new NuevoMotorV12());
```



[https://gist.github.com/](https://gist.github.com/2006100)



[https://gist.github.com/](https://gist.github.com/2006100)